Step 5 - Supabase Implementation

▼ 5.1 Overview

The GymSpot backend infrastructure is built using **Supabase**, which provides a managed PostgreSQL database, authentication services, and API generation.

Supabase allows the application to maintain a scalable backend architecture while simplifying common backend concerns such as:

- authentication
- database access
- row-level security
- API generation

The database schema defined in the previous step is implemented directly in PostgreSQL using Supabase.

Additionally, Supabase Authentication is used to manage user accounts securely.

▼ 5.2 Authentication Strategy

GymSpot relies on **Supabase Auth** to manage authentication and user identity.

Supabase automatically maintains the internal table:

```
auth.users
```

Instead of duplicating authentication data, the application uses a **profiles table** that extends the information associated with each authenticated user.

This approach ensures that:

- authentication remains secure and managed by Supabase

- application-specific user data is stored separately
- user data can be extended without modifying authentication structures

▼ 5.3 Profiles Table

The profiles table stores additional user information that is not part of the authentication system.

profiles

field	type	description
id	uuid	references auth.users.id
username	text	display name
avatar_url	text	profile picture
fitness_goal	text	user goal
created_at	timestamp	profile creation

Relationship:

```
profiles.id → auth.users.id
```

This establishes a **one-to-one relationship** between authentication records and application profiles.

▼ 5.4 Core Tables

The following tables implement the relational structure defined in the database design phase.

▼ routines

Stores user-created workout routines.

field	type
id	uuid
user_id	uuid
name	text
description	text
created_at	timestamp

Relationship:

```
routines.user_id → profiles.id
```

▼ exercises

Stores exercise information.

Exercises may originate from an external API but are stored locally to enable filtering and faster querying.

field	type
id	uuid
name	text
muscle_group	text
equipment	text
description	text

▼ routine_exercises

Links exercises to routines.

field	type
id	uuid
routine_id	uuid
exercise_id	uuid
sets	integer
reps	integer
rest_seconds	integer

Relationships:

```
routine_exercises.routine_id → routines.id
routine_exercises.exercise_id → exercises.id
```

▼ workouts

Represents an actual training session performed by the user.

field	type
id	uuid
user_id	uuid
routine_id	uuid
started_at	timestamp
finished_at	timestamp

Relationships:

```
workouts.user_id → profiles.id
workouts.routine_id → routines.id
```

▼ workout_sets

Stores the performance data of each exercise within a workout.

field	type
id	uuid
workout_id	uuid
exercise_id	uuid
weight	numeric
reps	integer

Relationships:

```
workout_sets.workout_id → workouts.id
workout_sets.exercise_id → exercises.id
```

▼ gyms

Stores gym locations used in the map discovery feature.

field	type
id	uuid
name	text
address	text
latitude	numeric

field	type
longitude	numeric

This table may be partially populated using external APIs such as Google Places.

▼ favorites

Stores user-specific saved items.

field	type
id	uuid
user_id	uuid
exercise_id	uuid
gym_id	uuid

Relationships:

```
favorites.user_id → profiles.id
favorites.exercise_id → exercises.id
favorites.gym_id → gyms.id
```

▼ 5.5 Row Level Security (RLS)

Supabase uses PostgreSQL Row Level Security to ensure that users can only access their own data.

Example policies:

routines

Users can only access routines they created.

```
auth.uid() = user_id
```

workouts

Users can only view or modify their own workouts.

```
auth.uid() = user_id
```

This ensures that all personal data remains securely isolated between users.

▼ 5.6 API Access

Supabase automatically exposes REST and RPC endpoints for database access.

The application interacts with the database through:

- Supabase client SDK
- secure authentication tokens
- filtered queries based on the authenticated user

This approach simplifies backend implementation while maintaining strong security guarantees.